SOFTWARE DEVELOPMENT PLAN

CampusTwin EnergyMind

Digital Twin of a University Building for
Occupancy-Aware Energy and Indoor Environmental Monitoring

Document Item	Value
Document ID	CTEM-SDP-001
Version	1.0
Date	20 September 2026
Project Type	B.Tech Major Project
University	IILM University, Greater Noida
Academic Session	2026–27
Development Status	Implementation-ready academic roadmap
Primary Baseline	SRS v1.0
Team	Purnima Giri • Yash Yadav • Piyush Sarkar • Yash Jaiswal

Prepared for academic development, evaluation and demonstration

Document Control and Source Alignment

Source	Version / Date	Role in Development Plan
SRS	CampusTwin_EnergyMind_SRS(5).pdf	v1.0, 18 September 2026
SAD	CampusTwin_EnergyMind_System_Architecture_Document(4).pdf	v1.0, 20 September 2026
UI/UX	CampusTwin_EnergyMind_UI_UX_Design_Document_Final.pdf	v1.0, 20 September 2026
PRD	PRD - Digital Twin for University Building Energy & Resource Management(5).pdf	current project baseline
Phase-I	26P_3CSE376(2).docx	Phase-I report, May 2026

Source-priority rule: the SRS controls implementation requirements; the SAD controls technical boundaries; the UI/UX document controls screen and interaction structure; the PRD controls product goals and MVP framing; the Phase-I report provides concept/research and future technology direction.

Scope clarification carried into this plan: the PRD defines a lightweight 2D MVP, while the Phase-I report describes a 3D Digital Twin. The SRS and SAD retain the final 2D/3D renderer as TBD. The plan therefore keeps the spatial data model and UI shell renderer-neutral.

Table of Contents

Section	Section	Section
1. Development Overview	2. Development Team Structure	3. Development Phases
4. Database Development	5. Building and Room Data Implementation	6. Backend Development Roadmap
7. API Development Plan	8. Authentication and RBAC	9. Telemetry Simulator
10. Energy-Waste Detection	11. Frontend Development Roadmap	12. Digital Twin Implementation
13. Frontend–Backend Integration	14. Real-Time Data Update Strategy	15. Testing Strategy
16. Performance Testing	17. Security Testing	18. Bug Management
19. Integration Milestones	20. Dependency Map	21. Development Priority
22. MVP Roadmap	23. Milestone Definitions	24. Definition of Done
25. Git and Version Control	26. Environment Management	27. Deployment Plan
28. Monitoring and Logging	29. Release Checklist	30. Risk-Based Development Plan
31. Development Timeline	32. Parallel Development Plan	33. Final End-to-End Flow
34. Requirement Traceability	35. Final Development Readiness Checklist	

1. DEVELOPMENT OVERVIEW

The development objective is to turn the approved CampusTwin EnergyMind requirements into a working web-based monitoring and decision-support prototype. The implementation must connect the building hierarchy, simulated or available telemetry, processing rules, alerts, recommendations and historical analytics through a practical layered architecture.

Development Approach

• Foundation first: repository, environments, database schema, backend skeleton and React shell.

• Vertical development: complete usable slices from PostgreSQL → FastAPI → React → UI → test.

• Simulation first: synthetic telemetry enables end-to-end development before hardware availability.

• Early integration: connect real frontend/backend/database flows before all screens are finished.

• Continuous testing: unit, API and integration checks begin as soon as each module exists.

• MVP discipline: future prediction, BMS control, federation, VR/AR and native mobile are not MVP work.

• No over-engineering: use the modular backend architecture; do not introduce microservices, Kubernetes or an enterprise event bus.

Confirmed Technical Baseline

Layer	Baseline	Status / Boundary
Frontend	React.js	Confirmed
Backend	Python + FastAPI	Confirmed
Database	PostgreSQL	Confirmed
Spatial MVP	SVG / JSON floor layouts	Supported; renderer decision remains TBD
Simulation	Python-based simulator	Confirmed capability; exact implementation details TBD
IoT	ESP32 + MQTT	Optional / Phase-I-supported when hardware and permissions exist
Analysis	Python; Scikit-learn	Future / Proposed for occupancy prediction
API access	Frontend through FastAPI	Confirmed; no direct browser-to-DB access

MVP Boundary

The current SRS baseline contains three buildings and 89 mapped rooms. The application ends at monitoring and human decision support; it does not issue actuator commands to HVAC, lighting, breakers or other critical infrastructure.

Development Flow

[Requirements Freeze]

↓

[Data Foundation]

↓

[Backend + Simulator]

↓

[Core Monitoring]

↓

[Waste Intelligence]

↓

[UI + Analytics]

↓

[Integrated Testing]

↓

[Demo Release]

2. DEVELOPMENT TEAM STRUCTURE

The team should work as a cross-functional student development group. The responsibility areas below are ownership areas, not rigid one-person assignments. A member may own a feature while another performs review/testing.

Area	Primary Responsibilities	Main Deliverables	Key Dependencies
Frontend	React setup; shell; dashboard; building/floor/room UI; alerts; recommendations; analytics; responsive states.	Working screens and reusable components.	API contracts, UI/UX baseline, building data.
Backend	FastAPI; authentication; RBAC; validation; services; waste rules; recommendations; analytics APIs.	Modular application/API layer.	Schema, requirements, simulator payload shape.
Database	PostgreSQL; schema; relationships; constraints; indexes; seed data; telemetry storage.	Migration/schema + seed scripts.	SRS data dictionary, building mapping.
Data / Simulation	Synthetic occupancy, energy, environment, HVAC and lighting streams; realistic usage patterns; waste scenarios.	Repeatable telemetry generator.	Room mapping, common telemetry schema.
Testing / QA	Unit, API, integration, UI, acceptance, performance and security checks; bug lifecycle.	Test suite, test evidence, defect log.	Each completed vertical slice.
Integration / Release	Environment configuration, end-to-end validation, demo setup, documentation and release checklist.	Demo-ready build and release package.	All P0 features and test gates.

Suggested Working Cadence

• Daily: short task sync, blocker identification and update of task/bug status.

• Feature completion: developer self-test → peer review → automated/manual tests → integration check.

• End of each phase: demo the deliverable against its exit criteria before moving to dependent work.

• Weekly: review scope, risks, unresolved TBD decisions and remaining P0 work.

3. DEVELOPMENT PHASES

Phase 0 — Project Preparation

Task	Output	Exit Check
Review all five source documents	Shared implementation baseline	Team can identify confirmed, proposed, TBD and future-scope items.
Freeze MVP requirements	P0 list + accepted room mapping	No unapproved feature enters active MVP work.
Record unresolved decisions	Decision log	2D/3D renderer and other TBD items explicitly tracked.
Create Git repository and workflow	Branches + README + .gitignore	All members can clone, branch and commit.
Configure local environments	React/FastAPI/PostgreSQL/Python simulator ready	Each core service starts locally.
Define coding conventions	Short team convention note	Naming, formatting and review rules agreed.

Phase 1 — Project and Environment Setup

Track	Tasks	Deliverable
Frontend	Initialize React; app shell; route/view structure only where required; reusable layout.	React application starts with base shell.
Backend	Initialize FastAPI; configuration; modular packages; API boundary.	FastAPI service starts and exposes health/basic service path if approved.
Database	Create PostgreSQL development DB; connection configuration; migration/schema mechanism.	Backend connects successfully.

Track	Tasks	Deliverable
Repository	README; environment example; ignore rules; basic branch policy.	Reproducible project setup.

Phase 2 — Data Foundation

Create the spatial hierarchy and telemetry persistence before building complex screens. Seed all three buildings, their configured floors and the exact room IDs. Then insert representative telemetry records and verify the relationships.

Phase 3 — First Vertical Slice

Complete one representative flow end-to-end: authenticated request → building/floor/room retrieval → room telemetry retrieval → React room view → validation and error handling. This establishes the integration pattern before the remaining features are expanded.

4. DATABASE DEVELOPMENT

The PostgreSQL model follows the SRS/SAD entities. Historical data is represented by timestamped telemetry/state records; a separate generic “HistoricalData” table is not required by the source baseline.

Entity	Purpose	Primary Key	Important Foreign Keys
User	Authenticated account and role context.	user_id	Role is a configured enum/value; exact auth-store mechanism remains implementation detail.
Building	Top-level spatial unit.	building_id	None
Floor	Building subdivision and spatial layout reference.	floor_id	building_id → Building
Room	Primary monitoring unit.	room_id	floor_id → Floor
Occupancy Stream	Timestamped aggregate presence/count.	stream_id + timestamp or approved unique record key	room_id → Room
Environmental Data	Temperature, humidity and CO■.	approved record identifier	room_id → Room
Energy Stream	Timestamped active power and energy.	stream_id + timestamp or approved unique record key	room_id → Room
Resource State	Timestamped HVAC/lighting state.	state_id	room_id → Room
Alert	Detected event requiring attention.	alert_id	room_id → Room
Recommendation	Evidence-linked decision-support output.	approved recommendation identifier	room_id → Room

Key Field Requirements

Entity	Key Fields	Validation / Notes
Building	ID, Name, Total Floors, Address, Gross Area	ID/name required; address/area optional where source data exists.
Floor	ID, Building_ID, Floor Number, Floor Map SVG/JSON	Parent building must exist; layout format must be valid.
Room	ID, Floor_ID, Room Number, Room Type, Max Capacity, Standby Threshold	Room ID must match supplied mapping; type/capacity/threshold may be TBD.
Occupancy	Stream_ID, Room_ID, Timestamp, IsOccupied, OccupantCount	Count ≥ 0; timestamp valid/not future.
Energy	Stream_ID, Room_ID, Timestamp, ActivePower, EnergyConsumed	Power/energy ≥ 0; timestamp valid/not future.
Resource State	State_ID, Room_ID, Timestamp, LightingState, HVACState	Lighting: ON/OFF/Dimmed; HVAC: Active/Idle/OFF.
Alert	Alert_ID, Room_ID, Timestamp, Type, Severity, Status, EstimatedWastedkWh	Status Active/Resolved; wasted kWh non-negative when present.
Recommendation	Issue, Room, Supporting Metric, Suggested Action, Severity, Timestamp, Status	Evidence is required before generation.

Indexes and Constraints

- Primary keys on major entities and foreign keys for spatial/telemetry relationships.
- Indexes on room/stream + timestamp for telemetry retrieval.
- Indexes supporting alert room/status/timestamp filtering.
- Indexes on building/floor parent relationships for spatial lookup.
- Parameterized database operations to reduce injection risk.
- Constraints for non-negative counts/energy, valid parent relationships and allowed resource states.

5. BUILDING AND ROOM DATA IMPLEMENTATION

The current MVP contains exactly 89 mapped rooms. Room ranges are inclusive and must be stored/rendered exactly as supplied. No additional room IDs are introduced.

Building	Floor	Room Range	Count
SVH — Swami Vivekanand Hall	Ground	101–112	12
SVH — Swami Vivekanand Hall	Second	201–213	13
Engineering Block	Ground	101–108	8
Engineering Block	First	201–208	8
Engineering Block	Second	301–308	8
Engineering Block	Third	401–408	8
Law Block	Ground	101–108	8

Building	Floor	Room Range	Count
Law Block	First	201–208	8
Law Block	Second	301–308	8
Law Block	Third	401–408	8

Naming note: the current project presentation label is “Swami Vivekanand Hall (SVH)”. The SRS/SAD source text contains the older “Sir Vishveshwarya Hall” label. This development plan preserves the supplied room/floor mapping and treats the display-name update as a presentation/data-label clarification, not as a change to room structure.

Implementation Sequence

• Insert building records.

• Insert configured floor records under each building.

• Insert exact room records under each floor.

• Attach room metadata only where the source supplies it; otherwise retain fields as optional/TBD.

• Attach SVG/JSON floor-layout references where available.

• Validate building → floor → room relationships.

• Test that each configured room appears exactly once and no unlisted room appears.

• Verify navigation through the same API consumed by the React UI.

Extensibility Rule

Building-specific navigation logic should not be hard-coded into React. New buildings should be introduced through Building/Floor/Room configuration and corresponding telemetry streams so that the same application structure can consume them.

6. BACKEND DEVELOPMENT ROADMAP

Use one modular FastAPI backend. The modules below are logical boundaries inside the application, not separate services.

Module	Implementation Tasks	Done When
Authentication	Credential verification; session/token context; logout; expiry handling.	Protected APIs reject unauthenticated access and valid users receive the correct role context.
Building	Buildings, floors, rooms, metadata and spatial references.	Three-building hierarchy is retrievable and validated.
Telemetry	Occupancy, energy, environment and resource-state ingestion/retrieval.	Valid payloads persist and invalid payloads are rejected/quarantined.
Processing	Validation, filtering, aggregation, current-state calculation.	Business services consume normalized validated data.
Waste Detection	Occupancy/resource/energy correlation; threshold + duration evaluation.	Qualifying scenarios generate one logical alert.
Alerts	Retrieval, filtering, status lifecycle and permitted actions.	Authorized users can review supported alert information/actions.
Recommendations	Evidence-linked rule-based recommendations.	Required fields are produced only with valid evidence.
Analytics	24h/7d/30d historical aggregation; occupancy-energy comparisons.	Authorized queries return bounded, chart-ready data.
Simulation	Simulator control only where supported; ingestion coordination.	Synthetic stream reaches normal processing path.

Recommended Backend Request Path

Backend Request Processing

[Client Request]

↓

[Authentication / RBAC]

↓

[Input Validation]

↓

[Business Service]

↓

[Data Access / Processing]

↓

[PostgreSQL or Decision Output]

↓

[Structured Response]

↓

[React UI]

Implementation Rules

• Backend authorization is authoritative; frontend visibility is not a security boundary.

• Keep validation close to ingestion and before valid-state persistence.

• Keep data access parameterized and transaction-safe.

• Return structured errors without internal stack traces.

• Do not fabricate zero values when telemetry is missing.

• Generate recommendations only when supporting evidence is valid.

• Keep processing and analytics in the modular backend rather than creating microservices.

6.1 Backend Build Order

Order	Build Slice	Dependencies	Test Gate
1	Configuration + DB connection	PostgreSQL setup	Connection and configuration tests.
2	User/auth context	User data + security configuration	Valid/invalid login and protected endpoint tests.
3	Building/floor/room services	Spatial schema + seed data	Hierarchy and room-ID preservation tests.
4	Telemetry validation + ingestion	Telemetry schema + room mapping	Invalid count, energy, timestamp and duplicate tests.
5	Current-state retrieval	Validated telemetry	Room detail API test.
6	Waste detection	Occupancy + energy + resource state	Zero occupancy + above-threshold duration test.
7	Alerts	Waste engine	Alert creation/filter/status tests.
8	Recommendations	Waste/rule evidence	Required-field and evidence tests.
9	Historical analytics	Timestamped history	24h/7d/30d query tests.
10	Simulation integration	Ingestion path	Simulator-to-database test.

Processing Order for a Telemetry Record

Development Flow

[Receive]

↓

[Identify Room/Stream]

↓

[Validate]

↓

[Deduplicate]

↓

[Persist]

↓

[Update Current State]

↓

[Evaluate Rules]

↓

[Expose via API]

7. API DEVELOPMENT PLAN

The source documents define API categories and behavior but do not finalize exact endpoint paths, HTTP methods, payload schemas or error codes. Therefore, endpoint naming and detailed contract syntax are implementation decisions and must be recorded as Proposed/TBD rather than presented as finalized source requirements.

API Category	Purpose	Typical Input	Output	Auth / Validation	Consumer
Authentication	Login/logout/session operations.	Credentials/session data	Session result/status	Protected as applicable; generic credential errors	Login/session UI
Building/Floor/Room	Retrieve configured spatial structure.	Building/floor/room identifiers	Metadata + room context	Role-dependent; unknown IDs handled	Dashboard/building/room UI
Occupancy	Ingest/retrieve current and historical occupancy.	Room, timestamp, occupied flag, count	Accepted state/history	Protected ingestion; count/timestamp validation	Room/dashboard/analytics
Energy	Ingest/retrieve kW/kWh.	Room, timestamp, power/energy	Accepted state/history	Protected ingestion; non-negative values	Room/dashboard/analytics
Environment	Ingest/retrieve temperature, humidity, CO■.	Room, timestamp, values	Accepted readings	Range/data-quality validation	Room/analytics
Resource State	Ingest/retrieve HVAC and lighting state.	Room, timestamp, states	Accepted state/history	Enum validation	Room/dashboard
Alerts	Retrieve/create/resolve supported alert records.	Room/type/evidence/status	Alert/status	Role-dependent; lifecycle validation	Alerts UI

API Category	Purpose	Typical Input	Output	Auth / Validation	Consumer
Recommendations	Retrieve decision-support records.	Condition/evidence/status	Recommendation record	Evidence required	Recommendations UI
Historical Analytics	Return 24h/7d/30d trends.	Room/building/date range	Aggregated time series	Role-dependent; bounded ranges	Analytics UI
Simulation	Start/stop/configure synthetic telemetry where supported.	Simulation configuration	Simulation status	Admin/Proposed; configuration validation	Project/demo tooling

API Contract Checklist

• Define request/response schema before frontend integration.

• Define authentication requirement and permitted roles per operation.

• Define validation rules and a stable structured error shape.

• Define not-found, forbidden, unauthorized and dependency-failure behavior.

• Document timestamp and unit conventions.

• Keep exact paths and methods marked Proposed until the team finalizes them.

8. AUTHENTICATION AND RBAC DEVELOPMENT

Role	Core Development View
Facility / Building Administrator	Building-wide visibility, utilization trends, alerts/anomalies and SRS-permitted administrative functions.
Energy / Resource Manager	Occupancy-energy correlation, waste alerts, historical analysis and recommendations.
Campus Operations Staff	Room/resource status, operational waste locations and inspection-oriented alerts.
Faculty / Student	Read-only room availability/current occupancy where enabled; other metrics remain role-dependent/proposed.

Implementation Sequence

• Create user and role representation.

• Implement secure password handling; never store plaintext passwords.

• Implement authenticated session/token context.

• Create backend role-verification dependency/service.

• Protect administrative, ingestion and role-restricted APIs.

• Implement protected UI views/actions consistent with the SRS permission matrix.

• Implement session expiry and re-authentication behavior.

• Implement logout and verify protected-page behavior after logout.

• Test forbidden access independently of UI hiding.

Security Boundary

The backend/API is the final authorization boundary. A hidden or disabled button is only a usability behavior. Direct requests to protected endpoints must still be rejected when the user's role is not authorized.

Required Authentication States

• Loading/submitting

• Invalid credentials

• Authenticated

• Forbidden action

• Expired session

• Logged out

• Backend unavailable

9. TELEMETRY SIMULATOR DEVELOPMENT

The simulator is the primary enabler for software-first development. It must emit the same logical telemetry structure expected from future real sensor/IoT sources so that replacing synthetic values with selected ESP32/MQTT streams does not require redesigning the application.

Signal	Required Shape	Scenario Coverage
Occupancy	IsOccupied + aggregate occupant count + timestamp	Occupied, unoccupied, transition, missing.
Energy	Active power kW + energy kWh + timestamp	Normal, standby, high load, zero load.
Temperature	Value + timestamp	Normal variation, missing, invalid range.
Humidity	Value + timestamp	Normal variation, missing, invalid range.
CO■	Value + timestamp	Normal/high occupancy relationship, missing.
HVAC	Active / Idle / OFF + timestamp	Occupied normal, empty but active, OFF.
Lighting	ON / OFF / Dimmed + timestamp	Occupied, empty but ON, OFF.

Generation Logic

• Load configured building/floor/room records.

• Assign realistic university usage patterns by time-of-day and room scenario.

• Generate occupancy first; derive plausible environmental/resource/energy conditions from the scenario.

• Create deliberate waste scenarios for test coverage.

• Emit timestamps that are valid and not in the future.

• Send data through the normal ingestion path instead of writing directly to final-state tables.

• Log accepted/rejected records and validation reasons.

• Allow repeatable demo scenarios where practical.

Mandatory Demonstration Scenarios

• Empty room + HVAC ON for longer than the configured waste duration.

• Empty room + Lighting ON.

• Occupied room + normal load.

• High occupancy + corresponding energy/environment changes.

• Missing telemetry.

• Invalid negative energy/count.

• Future timestamp.

• Duplicate telemetry.

• Non-qualifying threshold or duration.

10. ENERGY-WASTE DETECTION DEVELOPMENT

The waste engine is a rule-based decision-support component. The SRS requires zero occupancy plus load above the configured standby threshold continuously for the configured duration before an energy-waste alert is created. The PRD gives a 15-minute example; exact thresholds remain configurable/TBD where the SRS does not finalize them.

Development Flow

[Validated Occupancy]

↓

[Validated Energy/Resource State]

↓

[Room Match]

↓

[Occupancy Check]

↓

[Threshold Check]

↓

[Duration Check]

↓

[Alert Generation]

↓

[Estimated kWh]

↓

[Persist]

↓

[Recommendation Evidence]

Rule Implementation Tasks

• Normalize the latest valid room state.

• Determine whether occupancy is zero/empty using valid data.

• Evaluate HVAC and lighting operational states.

• Compare relevant load against the configured standby threshold.

• Track continuous qualifying duration; do not trigger on a single transient sample.

• Generate a logical alert when all required conditions are satisfied.

• Calculate estimated wasted kWh only when the source data supports the calculation.

• Persist the alert with room, type, timestamp, severity, status and available evidence.

• Expose the alert through the normal API.

• Generate a recommendation only when valid evidence exists.

Test Matrix

Scenario	Expected Result
Empty + HVAC ON + above threshold + duration met	One qualifying waste alert is generated.
Empty + HVAC ON + threshold met but duration not met	No waste alert yet.
Occupied + normal load	No waste alert.
Missing occupancy	Do not infer zero; no unsupported waste alert.
Negative energy / invalid telemetry	Reject or quarantine; do not update valid state.
Duplicate record	No duplicate logical state/alert.

11. FRONTEND DEVELOPMENT ROADMAP

The React application should follow the approved UI/UX structure and use reusable components. The frontend consumes data through FastAPI and does not connect directly to PostgreSQL.

Order	Screen / Area	Core Work	Required States
1	Application Shell	Header, sidebar, main content, user/session area.	Loading shell where relevant; protected/unauthorized.
2	Login	Identifier, password, sign-in, generic errors, session expiry.	Default, validation, loading, invalid credentials, expired session.
3	Dashboard	Summary cards, building/floor context, spatial panel, alerts, quick analytics.	Loading, empty, stale, API error.
4	Building View	Building selector, floor selector, room rendering, legend, selection.	Map loading, no floor, map error, stale room.
5	Room Detail	Occupancy, environment, energy, HVAC, lighting, timestamp, 24h preview.	Metric unavailable/stale, loading, incomplete data.
6	Alerts	Filterable alerts, severity, duration/timestamp, wasted kWh, supported actions.	Loading, empty, error, unauthorized action.
7	Recommendations	Issue, room, evidence, suggested action, severity, timestamp, status.	Loading, empty, error.
8	Historical Analytics	24h/7d/30d; building/floor/room filters; occupancy-energy charts.	Loading, incomplete gaps, query error.

Reusable Component Set

• Header, sidebar, breadcrumb, building selector, floor selector.

• Metric card, status badge, room node/card, alert card/table, recommendation card.

• Spatial map/canvas, legend, chart container, tooltip, heatmap where supported.

• Toast/banner, skeleton, empty state, error state, retry control and confirmation dialog where required.

11.1 Frontend Implementation Rules

State Model

State	UI Behavior
Default / Success	Render validated data and timestamp.
Loading	Use local skeleton/spinner without blocking unrelated sections.
Empty	Explain that no data exists for the selected room/time range.
Partial / Stale	Show last valid timestamp and explicitly mark unavailable/stale metrics.
Error	Show controlled message and retry option; never show internal stack traces.
Unauthorized / Session Expired	Reject protected operation and direct the user to authentication as appropriate.

Data Visualization Rules

• Room states must not rely on color alone; pair color with text/icon/badge.

• Use explicit units for kW, kWh, °C, %, ppm and occupancy count.

• Show timestamps for current telemetry.

• Show visible gaps/incomplete-data states rather than connecting or inventing missing measurements.

• Use 24-hour, 7-day and 30-day views as defined by the source baseline.

• Keep chart payloads bounded through backend aggregation.

Responsive and Accessibility Gate

• Validate desktop layouts at 1920×1080 and 1366×768.

• Keyboard navigation and visible focus.

• Semantic labels and accessible form errors.

• Tooltips must not be the only way to reach information.

• Sufficient contrast and color-independent status communication.

• Logical tab order and accessible status announcements where needed.

12. DIGITAL TWIN IMPLEMENTATION

The spatial renderer is deliberately isolated from the building/telemetry model. This is the key architectural response to the source conflict: PRD/SRS support a 2D MVP, while Phase-I describes a 3D Digital Twin. The final renderer remains TBD.

Decision Path	Implementation	Boundary
If 2D is confirmed	SVG/JSON floor layouts; room boundaries; room status; clickable rooms; floor switching.	Directly supported by PRD/SRS; appropriate MVP path.
If 3D is confirmed later	Three.js / React Three Fiber renderer consuming the same building/room/telemetry model.	Phase-I-supported path; requires explicit team/guide confirmation.
Regardless of renderer	Building selector → floor selector → spatial view → room selection → room details.	Navigation and data model remain stable.

Renderer-Neutral Contract

• Input: building, floor, room geometry/layout metadata and latest valid room state.

• Output: selected room, visual status and navigation event.

• Renderer does not own business rules.

• Renderer does not calculate authoritative occupancy/energy state.

• Room IDs come from configuration/database and must not be changed by the visual layer.

• Telemetry freshness and unavailable state must remain visible in the room UI.

Floor/Room Flow

Development Flow

[Select Building]

↓

[Load Configured Floors]

↓

[Select Floor]

↓

[Load Layout]

↓

[Render Rooms]

↓

[Select Room]

↓

[Open Room Detail]

13. FRONTEND–BACKEND INTEGRATION

Integration Order	Dependency	UI Consumer	Required Test
Authentication	User/auth APIs	Login + protected shell	Valid/invalid login, logout, expiry.
Building/Floor/Room	Spatial schema/seed	Dashboard + Building View	All three buildings and exact room IDs.
Current Telemetry	Ingestion + state processing	Dashboard + Room Detail	Valid state appears with timestamp.
Alerts	Waste detection + alert persistence	Dashboard + Alerts	Qualifying alert appears.
Recommendations	Evidence/rules	Recommendations	Required fields/evidence.
Historical Analytics	Timestamped data + aggregation	Analytics	24h/7d/30d render.
Simulator	Common ingestion schema	Demo system	Simulator → DB → API → UI.
Error Handling	Structured backend errors	All affected widgets	Retry/empty/stale/forbidden states.

Integration Gate

Do not merge a feature as complete when only the frontend mock or only the backend endpoint works. A feature reaches an integration gate when its API contract, data source, UI state handling and relevant tests are connected.

End-to-End Data Path

Development Flow

[Simulator / REST / Optional ESP32-MQTT]

↓

[FastAPI Ingestion]

↓

[Validation + Processing]

↓

[PostgreSQL]

↓

[Waste / Analytics Services]

↓

[FastAPI APIs]

↓

[React UI]

14. REAL-TIME DATA UPDATE STRATEGY

The source documents require an ingestion-to-dashboard update target but do not finalize one single frontend real-time transport. The PRD mentions REST/WebSocket ingestion, while the SAD defines REST-style application APIs and optional MQTT for device telemetry. The development plan therefore avoids locking the project to an unapproved WebSocket implementation.

Implementation Decision Rule

• Use the source-supported application/API path first.

• Choose the final browser refresh mechanism only after the team confirms the implementation detail; record it as Proposed if not source-finalized.

• Measure the actual ingestion-to-UI latency against the SRS 2-second target.

• Do not let the UI depend on direct database polling.

Freshness and Failure Handling

Condition	System Behavior	UI Behavior
Fresh update	Store/serve latest valid state.	Update room/dashboard state and timestamp.
Delayed source	Retain last valid state.	Mark stale/unavailable; show last update time.
Missing telemetry	Do not substitute measured zero.	Show “Data unavailable” for the metric.
Invalid telemetry	Reject/quarantine; log reason.	Continue showing other valid metrics.
API failure	Return controlled error.	Localized error + retry.
Session expiry	Invalidate protected context.	Require re-authentication.

Acceptance Gate

The integration test must measure a representative state change from ingestion through backend processing to visible UI state and record the measured latency.

15. TESTING STRATEGY

Testing begins with the first vertical slice. The SRS acceptance criteria and test cases are the authoritative source for the major verification set.

Layer	What to Test	Minimum Evidence
Unit — Backend	Validation, business rules, waste detection, recommendation rules, aggregation.	Automated tests + pass record.
Unit — Frontend	Reusable components, status rendering, validation, role-aware visibility.	Component test results.
API	Valid/invalid requests, auth, RBAC, missing data, invalid IDs, error responses.	API test collection/results.
Integration	Frontend→API→backend→DB; simulator→ingestion→processing→DB→API→UI.	End-to-end scenario results.
UI	Login, navigation, building/floor/room, room details, alerts, recommendations, analytics.	Manual/automated UI checklist.
Acceptance	SRS AC-01…AC-18 and mapped test cases.	Signed/recorded acceptance matrix.

Core SRS Test Set

Test ID	Scenario	Expected Result
TC-01/02	Valid login, invalid login and logout	Authentication and session termination behave correctly.
TC-03	RBAC enforcement	Forbidden operation rejected without protected data exposure.
TC-04/05	Building/floor navigation and room-ID preservation	All configured mappings load; no extra rooms.
TC-07	Occupancy state/count	Valid states display; negative count rejected.
TC-08	Temperature/humidity/CO■	Valid values + timestamps display.

Test ID	Scenario	Expected Result
TC-09/10	Energy + HVAC/lighting	Stored/retrieved values match validated input.
TC-11	Waste detection	Qualifying condition creates one logical alert.
TC-12	Alert lifecycle	Supported status action persists correctly.
TC-13	Historical analytics	24h/7d/30d charts render for sufficient history.
TC-14/15	Simulator + invalid ingestion	Schema-valid data flows; malformed data is rejected/quarantined.
TC-17/18	Recommendations + no physical control	Evidence-linked recommendation appears; no actuator command exists.

15.1 Testing Gates by Development Stage

Stage	Entry	Exit Test Gate
Schema	DB schema created	Constraints, relationships and seed mapping validated.
API foundation	Backend service starts	Auth, spatial retrieval and error responses tested.
Telemetry	Ingestion path exists	Valid/invalid/duplicate/future records handled.
Core monitoring	Current-state APIs exist	Dashboard and room detail show correct values.
Waste intelligence	Validated telemetry exists	Waste scenarios pass; false trigger cases do not.
Analytics	History exists	24h/7d/30d queries and UI pass.
MVP	All P0 features integrated	Acceptance suite passes; no Critical/High unresolved bugs.
Release	Demo build available	Smoke + security + performance + end-to-end checks pass.

Test Evidence to Preserve

• Test case ID, date, environment and build/commit.

• Input data/scenario and expected result.

• Observed result and pass/fail.

• Screenshot/log/API evidence for important acceptance cases.

• Defect ID for failures and retest evidence after fixes.

16. PERFORMANCE TESTING

Requirement	Target	Validation Method
Dashboard update	Within 2 seconds after state-change ingestion	Timestamp ingestion and visible UI update under representative load.
30-day chart	≤ 1.5 seconds	Measure API/query + render time using representative 30-day dataset.
Telemetry scale	Up to 500 rooms at 5-second sampling	Controlled synthetic load; monitor ingestion errors and query responsiveness.
Availability	99% during 07:00–22:00	Measure during defined validation/deployment period.
Desktop support	1920×1080 and 1366×768	Manual visual and interaction checks.

These are validation targets from the source baseline. The development team must measure them; implementation should not claim compliance merely because the architecture is capable of supporting the target.

17. SECURITY TESTING

Area	Test
Authentication	Invalid credentials, logout, session expiry and protected page access.
Authorization	Attempt restricted API/UI actions using each role.
Input validation	Malformed IDs, negative counts/energy, invalid enums, future timestamps and bad environmental values.
Injection protection	Verify parameterized database operations and malicious input handling.
Sensitive exposure	Verify errors do not expose stack traces, credentials or unnecessary internal details.
Transport	Verify applicable deployed traffic uses TLS 1.3.
Privacy	Confirm no PII, facial images, biometric identifiers or individual identity tracking are stored.
Physical control boundary	Inspect APIs/UI for absence of actuator-control commands.

Proposed Security Safeguards

The SRS/SAD mark password hashing, secure secret storage, least-privilege accounts, secure headers, rate limiting where appropriate and security-event logging as proposed safeguards. They should be implemented where practical and recorded as implementation safeguards rather than new product requirements.

18. BUG MANAGEMENT AND FIXING PROCESS

Bug Lifecycle

[New]

↓

[Confirmed]

↓

[Assigned]

↓

[In Progress]

↓

[Fixed]

↓

[Retest]

↓

[Closed]

Severity	Definition	Typical Response
Critical	System unusable or major security failure.	Immediate fix; blocks release.
High	Major MVP feature broken.	Fix before MVP acceptance.
Medium	Important functionality partially affected.	Fix before release where practical; otherwise documented.
Low	Minor UI/usability issue.	Schedule according to time and impact.

Required Bug Record

- Bug ID
- Description
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Assigned member
- Status
- Fix/commit
- Retest result

19. INTEGRATION MILESTONES

Milestone	Completion Condition
M1	Frontend + backend basic connection works.
M2	PostgreSQL + backend fully connected and tested.
M3	Building/floor/room navigation works with exact mapping.
M4	Simulator → backend → database works.
M5	Live/current room telemetry is visible in the frontend.
M6	Waste detection → alerts works.
M7	Recommendations work with evidence.
M8	Historical analytics works for 24h/7d/30d.
M9	Role-based workflow works across supported roles.
M10	MVP test and acceptance suite completed.
M11	Demo/deployment environment passes smoke and end-to-end checks.

20. DEPENDENCY MAP

Task	Depends On	Blocks
Repository/environment	Team agreement	All implementation
Database schema	SRS data dictionary	Backend models/services
Seed building/room data	Schema	Spatial APIs + frontend navigation
Backend spatial APIs	Schema + seed	Building/floor/room UI
Telemetry schema	SRS/SAD data model	Simulator + ingestion
Simulator	Room mapping + telemetry schema	Waste detection + realistic integration tests
Current-state APIs	Validated telemetry	Dashboard + room detail
Waste detection	Occupancy + energy + resource state	Alerts
Alerts	Waste detection + persistence	Alert UI
Recommendations	Rule evidence	Recommendation UI
Historical analytics	Historical telemetry	Analytics UI

Task	Depends On	Blocks
Authentication/RBAC	User model + backend	Protected screens/actions
UI integration	API contracts	End-to-end testing
Performance testing	Representative integrated build	Release readiness
Security validation	Authenticated integrated build	Release readiness

Parallelization

Database/backend foundation, React component work, simulator development and test-case preparation can proceed in parallel after the common data contract and room mapping are frozen. They converge at API integration and end-to-end testing.

Development Flow

[Common Data Contract]

↓

[Track A: DB + Backend]

↓

[Track B: React UI]

↓

[Track C: Simulator]

↓

[Track D: QA/Test Data]

↓

[Convergence: Integration]

↓

[Acceptance]

21. DEVELOPMENT PRIORITY

Priority	Implementation Scope
P0 — Must Have	Authentication; RBAC; three-building/floor/room mapping; occupancy; energy; HVAC/lighting; simulator; waste detection; alerts; dashboard; room details; historical analytics; core validation/error handling.
P1 — Important	Environmental metrics where telemetry is available; richer stale/data-quality handling; role-aware operational alert actions where supported; responsive/accessibility completion; performance/security validation; complete release evidence.
P2 — Optional / Proposed	ESP32/MQTT integration where hardware/network/permissions exist; exportable summaries; simulator control UI; selected visualization enhancements; other explicitly Proposed implementation choices.
P3 — Future Scope	Advanced occupancy prediction; predictive maintenance; BMS/BACnet/Modbus integration and physical control; campus-wide federation; timetabling integration; carbon tracking; native mobile; VR/AR.

22. MVP ROADMAP

MVP Roadmap

[MVP Foundation
Setup + DB + Backend]

↓

[MVP Data Pipeline
Simulator + Ingestion]

↓

[MVP Core Monitoring
Buildings + Telemetry]

↓

[MVP Intelligence
Waste + Alerts + Recommendations]

↓

[MVP Visualization
Dashboard + Spatial + Room]

↓

[MVP Analytics
24h/7d/30d]

↓

[MVP Security
Auth + RBAC]

↓

[MVP Testing
Unit/API/Integration/UI/Acceptance]

↓

[MVP Deployment
Demo Environment]

P3 items must not be allowed to delay the P0 path. If a P2 feature consumes time needed for acceptance, pause it and return effort to P0 defects and test gaps.

23. MILESTONE DEFINITIONS

Milestone	Objective	Tasks	Dependencies	Exit Criteria
Foundation Ready	Make implementation reproducible.	Repo, environments, conventions, decision log.	Requirements review	Every member can start core services.
Data Ready	Make spatial/telemetry storage trustworthy.	Schema, constraints, seed mapping, sample data, indexes.	Foundation	All 89 rooms validated; DB tests pass.
Backend Core Ready	Expose protected application services.	Auth, RBAC, spatial APIs, telemetry validation/state.	Data Ready	API tests pass for P0 flows.
Simulation Ready	Create complete software data stream.	Scenario generator + ingestion.	Backend telemetry	Simulator produces schema-valid data through normal path.
Monitoring Ready	Expose current room state.	Dashboard, spatial view, room details.	APIs + UI shell	Building/floor/room + current telemetry flow works.
Intelligence Ready	Detect and explain waste.	Waste engine, alerts, recommendations.	Telemetry + state	Waste scenarios pass and false cases do not.
Analytics Ready	Show historical relationships.	24h/7d/30d queries + charts.	History	Charts render with correct units/gaps.
MVP Accepted	Verify source-defined MVP.	Acceptance suite, security, performance, UI review.	All P0 integrated	No Critical/High unresolved; acceptance evidence complete.
Demo Ready	Run reliable academic demonstration.	Deployment/configuration/smoke test.	MVP Accepted	End-to-end demo succeeds on chosen environment.

Milestone Rule

A milestone is complete only when its deliverables work in the integrated environment and the stated test/exit criteria are satisfied. Code existence alone is not completion.

24. DEFINITION OF DONE

- Requirement is implemented against the approved baseline.
- Code follows the agreed project structure and conventions.
- Frontend/backend integration works where applicable.
- Database changes are applied, repeatable and tested.
- Validation is implemented.
- Authorization is implemented where required.
- Loading, empty and error states exist where applicable.
- Unit/API tests pass; integration tests pass where applicable.
- UI behavior matches the approved UI/UX baseline.
- Acceptance criteria are satisfied.
- No Critical or High unresolved defect remains for the feature/release gate.
- Documentation and relevant test evidence are updated.
- Code is committed to the repository and review/merge is complete.
- Feature is demonstrated successfully in the target environment.

25. GIT AND VERSION CONTROL PLAN

Practice	Simple Student-Team Rule
Main branch	Stable, demonstration-ready code only.
Development branch	Integration branch for completed feature work before release.
Feature branches	One feature/bug per branch, e.g. feature/room-detail or fix/telemetry-validation.
Commits	Short imperative messages with a clear scope; avoid unrelated changes.
Pull requests	At least one teammate reviews functional changes before merge.
Testing before merge	Run relevant unit/API/integration checks and record failures.
Merge	Merge only after review + test gate; resolve conflicts deliberately.
Tags	Tag milestone/demo releases, e.g. v0.1-foundation, v0.2-data, v1.0-mvp.

Recommended Repository Layout

The SAD labels its backend folder structure as recommended rather than finalized. A practical organization is: frontend/ for React; backend/ for FastAPI modules such as auth, api, schemas, services, ingestion, analytics, waste_detection, recommendations, data_access and configuration; database/ for schema/migrations/seed assets; simulator/ for Python generation; tests/ for shared test material; docs/ for project documents.

26. ENVIRONMENT MANAGEMENT

Environment	Arrangement	Purpose
Development	Local React + FastAPI + PostgreSQL + Python simulator.	Daily implementation and debugging.
Testing	Controlled test database/data with integrated services.	Repeatable unit/API/integration/acceptance validation.
Demonstration / Production-like	React static build/served frontend + FastAPI + PostgreSQL + simulator; optional ESP32/MQTT.	Academic demonstration.

The architecture does not finalize a cloud provider. Any hosting provider selected later must be marked Proposed/TBD until the team confirms it. The local demonstration path is explicitly supported and should be maintained as the fallback demo environment.

27. DEPLOYMENT PLAN

• Build the React frontend.

• Configure frontend environment variables/API base configuration.

• Serve/deploy the frontend in the selected environment.

• Deploy/start the FastAPI backend.

• Configure PostgreSQL and apply the approved schema.

• Seed building/floor/room mapping and controlled demonstration data.

• Configure simulator and its ingestion target.

• Configure secrets and security settings outside source code.

• Run smoke tests for login, building navigation and telemetry.

• Run simulator → ingestion → processing → DB → API → UI end-to-end test.

• Verify stale/error handling and role restrictions.

• Prepare the demonstration script and rollback/fallback local run.

Local Demonstration Path

Development Flow

[Start PostgreSQL]

↓

[Start FastAPI]

↓

[Start React]

↓

[Start Simulator]

↓

[Open Browser]

↓

[Login]

↓

[Select Building/Floor/Room]

↓

[Trigger Waste Scenario]

↓

[Show Alert + Recommendation + Analytics]

28. MONITORING AND LOGGING

Area	Observe	Response
API health	Service availability/health status.	Investigate process/deployment failure.
Database	Connection failures and query errors.	Log dependency failure; return safe service error.
Telemetry ingestion	Accepted/rejected records, source status, latest timestamp.	Show stale/offline state; inspect source/validation.
MQTT	Connection/message receipt when enabled.	Retry/log failure; never fabricate telemetry.
Authentication	Failed login/forbidden requests.	Record security event without credentials.

Area	Observe	Response
Validation	Invalid values, room IDs, timestamps, duplicates.	Record reason; reject/quarantine.
Waste engine	Rule evaluation and persistence failures.	Log failure; preserve evidence.
Performance	Ingestion-to-UI latency and chart time.	Compare against targets and optimize queries/UI.

The source architecture explicitly states that lightweight application logs and health checks are sufficient for the prototype. Centralized enterprise observability infrastructure is not required for the current MVP.

29. RELEASE CHECKLIST

Area	Release Gate
Functional	Login, RBAC, all three buildings, configured floors, exact room IDs, occupancy, energy, HVAC, lighting, environment where available, waste detection, alerts, recommendations and analytics all verified.
Technical	Database, FastAPI, simulator and React integration work end-to-end.
Security	Protected APIs, authentication, no PII, no physical-control commands, TLS 1.3 where applicable.
UI/UX	Approved UI/UX structure, loading/empty/error states, desktop layouts and accessibility basics verified.
Data Quality	Missing telemetry is not shown as zero; invalid/duplicate/future data handling verified.
Evidence	Acceptance results, test evidence, bug status and build/commit are recorded.

30. RISK-BASED DEVELOPMENT PLAN

Risk	Impact	Probability	Mitigation	Owner
Sensor availability	High	Medium	Use simulator first; integrate selected ESP32 streams only when hardware/permissions exist.	Data/IoT
Data quality / noisy telemetry	High	Medium	Validation, filtering, deduplication and explicit stale/unavailable states.	Backend + QA
Network connectivity	Medium/High	Medium	Local simulator path, retries/reconnection where supported, stale indicators.	Integration
Building access	Medium	Medium	Use available floorplan assets and simulation until access is approved.	Project Lead
Hardware cost	Medium	Medium	Use selected hardware and simulation for remaining rooms.	Data/IoT
Incomplete telemetry	High	High	Treat each metric independently; never infer zero.	Backend + Frontend
Floorplan availability	Medium	Medium	Use simplified SVG/JSON layouts for MVP.	Frontend
2D/3D decision	High	Medium	Keep renderer isolated; confirm with guide before committing to 3D.	Architecture/Team
Integration delay	High	Medium	Integrate early; enforce vertical-slice gates and weekly end-to-end demos.	All
Scope expansion	High	Medium	Protect P0; defer P2/P3 work.	Project Lead

Risk Response Priority

Risks that can block the software demonstration—sensor availability, incomplete telemetry, integration delay and the 2D/3D decision—should be handled before optional enhancements. The simulator and renderer-neutral architecture are the primary schedule protections.

31. DEVELOPMENT TIMELINE

The following is a suggested sequence rather than a fixed submission schedule. The team can compress or extend phases according to its academic calendar. Durations are working estimates for planning, not source requirements.

Phase	Suggested Duration	Main Tasks	Dependency	Milestone
0 Preparation	2–3 days	Document review, scope freeze, decision log, repo.	Source documents	Foundation Ready
1 Environment	3–4 days	React/FastAPI/PostgreSQL/simulator setup.	Phase 0	Environment Ready
2 Database + Seed	4–5 days	Schema, constraints, 89-room mapping, indexes.	Environment	Data Ready
3 Backend Core	6–8 days	Auth/RBAC, spatial APIs, telemetry validation/state.	Data Ready	Backend Core Ready
4 Simulator	4–6 days	Synthetic streams + scenarios + ingestion.	Telemetry contract	Simulation Ready
5 Core UI	6–8 days	Shell, login, dashboard, building/floor/room, room detail.	API contracts	Monitoring Ready
6 Intelligence	4–6 days	Waste rules, alerts, recommendations.	Validated telemetry	Intelligence Ready
7 Analytics	3–5 days	24h/7d/30d APIs and charts.	Historical data	Analytics Ready
8 Integration	4–6 days	End-to-end flows, error states, role checks.	Core features	Integrated MVP

Phase	Suggested Duration	Main Tasks	Dependency	Milestone
9 QA + Security + Performance	5–7 days	Acceptance, performance, security, UI/accessibility.	Integrated MVP	MVP Accepted
10 Release/Demo	2–3 days	Deployment, smoke tests, demo data/script, final docs.	MVP Accepted	Demo Ready

Schedule Protection Rule

If the schedule slips, remove or defer P2/P3 features first. Do not reduce validation, acceptance testing or the core simulator path merely to preserve optional visualization or integration work.

32. PARALLEL DEVELOPMENT PLAN

Track	Can Start When	Parallel Work	Converges At
A — Database + Backend	After scope/data model freeze	Schema, seed, auth, spatial APIs, telemetry validation.	API integration
B — Frontend	After UI/UX baseline + mock contract	Shell, components, login, dashboard, spatial shell, room detail.	API integration
C — Simulator	After telemetry schema + room mapping	Scenario generation and ingestion client.	Telemetry integration
D — QA	Immediately after requirements freeze	Test cases, fixtures, acceptance matrix, bug template.	Feature test gates
E — Documentation	Throughout	API notes, setup instructions, decision log, evidence.	Release package

Recommended Convergence Pattern

Do not wait for every track to finish. Converge one vertical slice at a time: spatial data → spatial API → React building/floor/room view → current telemetry → room detail → test. Then repeat for waste, alerts, recommendations and analytics.

Development Flow

[A+B+C+D]

↓

[First Vertical Slice]

↓

[Integration Test]

↓

[Second Slice: Waste]

↓

[Integration Test]

↓

[Third Slice: Analytics]

↓

[Full Acceptance]

33. FINAL END-TO-END DEVELOPMENT FLOW

Complete Development Pipeline

[Requirements]

↓

[Database]

↓

[Backend]

↓

[Simulator]

↓

[APIs]

↓

[Frontend]

↓

[Integration]

↓

[Waste Detection]

↓

[Alerts]

↓

[Recommendations]

↓

[Analytics]

↓

[Testing]

↓

[Bug Fixing]

↓

[Security Validation]

↓

[Deployment]

↓

[MVP Demonstration]

34. REQUIREMENT TRACEABILITY

Requirement / Feature	Development Task	Component	Test	Milestone	Status
FR-01–04 Authentication / access	User model, auth, RBAC, protected APIs/UI	Auth + shell	TC-01/02/03	M1/MVP Accepted	P0
FR-05–09 Digital Twin/navigation	Seed mapping + spatial APIs + renderer shell	Building View	TC-04/05/06	M3/M5	P0; 2D/3D TBD
FR-10–12 Occupancy	Occupancy ingestion/state/history	Dashboard/Room/Analytics	TC-07/13	M5/M8	P0
FR-13–15 Environment	Environmental ingestion/retrieval	Room/Analytics	TC-08	M5	P0 where data available
FR-17–20 Energy/resources	Energy + HVAC/lighting data path	Room/Dashboard	TC-09/10	M5	P0
FR-22–23 Waste detection	Threshold/duration correlation engine	Waste service	TC-11	M6	P0
FR-25–27 Alerts	Alert persistence/API/UI	Alerts	TC-12	M6	P0
FR-29 Historical analytics	Aggregation APIs + charts	Analytics	TC-13	M8	P0
FR-32 Simulation	Python simulator + normal ingestion	Simulator	TC-14	M4	P0
FR-33/35 Validation	Input validation/deduplication	Validation	TC-15	M4/M10	P0
FR-34 MQTT	Optional MQTT adapter	Ingestion	TC-16	P2	Optional/conditional
FR-36/37 Recommendations	Evidence-linked rule outputs	Recommendations	TC-17	M7	P0
FR-38 Control boundary	No actuator commands; review APIs/UI	Whole system	TC-18	M10	P0
NFR Performance	Measurement, query optimization, UI timing	Whole stack	TC-19	M10	Required validation
Security/Availability	TLS/RBAC/logging/uptime measurement	Whole stack	TC-20	M10/M11	Required validation

35. FINAL DEVELOPMENT READINESS CHECKLIST

Requirements

• PRD reviewed.

• SRS reviewed and accepted as primary implementation baseline.

• SAD reviewed.

• UI/UX baseline reviewed.

• Phase-I report reviewed.

• MVP scope frozen.

• 89-room mapping validated.

• TBD decisions recorded, especially 2D vs 3D.

Infrastructure

• Git repository ready.

• React environment ready.

- FastAPI environment ready.
- PostgreSQL ready.
- Python simulator environment ready.
- Environment variables/secrets strategy defined.

Development

- Folder/module structure defined.
- Database schema planned.
- API categories and contracts planned.
- Frontend screen/component structure planned.
- Responsibilities assigned.
- Dependencies and parallel tracks identified.

Testing

- SRS acceptance criteria mapped to tests.
- Core test fixtures/scenarios prepared.
- Bug lifecycle and severity definitions agreed.
- Definition of Done agreed.
- Performance and security measurement methods prepared.

Immediate Next 10 Actions

- Freeze the implementation baseline and decision log.
- Create/confirm Git repository and branch policy.
- Set up React, FastAPI, PostgreSQL and simulator environments.
- Create the PostgreSQL schema and migration/seed process.
- Insert the three buildings, configured floors and all 89 exact room IDs.
- Define the common telemetry payload/data contract.
- Build the first FastAPI spatial + telemetry vertical slice.
- Build the React shell, login and building/floor/room flow.
- Connect the simulator through the normal ingestion path.
- Run the first end-to-end test and use its defects to plan the next slice.

Final Implementation Principle

CampusTwin EnergyMind should be developed as a practical monitoring and decision-support prototype: Awareness → Investigation → Evidence → Human Decision Support. The software architecture remains modular, renderer-neutral and extensible, while the MVP stays focused on the documented three-building, 89-room baseline and avoids autonomous physical control.

Source Reference Note

This plan was prepared from the current project source set: CTEM-SRS-001, CTEM-SAD-001, CTEM-UIUX-001, the CampusTwin EnergyMind PRD, and the Phase-I Major Project Report. Where the sources differ, the plan preserves the documented status rather than silently converting a Proposed or TBD item into a confirmed requirement.